F.CSM302 Үйлдлийн систем — Бие даалт 1

The Dining-Philosophers Problem

Хуваарилалтын код: [КОД]
Гишүүд: [ГИШҮҮД]
Сурах бичиг: Silberschatz, Operating System Concepts, Ch.7 Project 11

Хураангуй

Dining-Philosophers бодлогыг monitor (нэг mutex + philosopher бүрт нэг condition variable) ашиглан Pthreads-ээр C хэл дээр хэрэгжүүлэв. Deadlock болон starvation-ийг ойлгоход зориулж thinking, hungry, eating төлөв ба fork эзэмшилтийг харуулдаг интерактив HTML UI бүтээв. UI нь Monitor болон Naive (зүүн→баруун) гэсэн хоёр алгоритмыг харьцуулж, Naive горимд deadlock-ийг илрүүлнэ. Автомат 45 тест (үндсэн, захын, стресс, алдааны) бүгд амжилттай.

1. Удиртгал

Зорилго: нийтлэг нөөц (fork) -ийн синхрончлол, deadlock-аас сэргийлэх аргыг хэрэгжүүлж, туршиж, дүрслэх. Хамрах хүрээ: 2–64 philosopher, тохируулах боломжтой thinking/eating хугацаа, seed.

2. Сурах бичгийн шаардлага

• Philosopher бүр thread, thinking ба eating үе ээлжилнэ (санамсаргүй хугацаа).

• Хөрш philosopher нар нэгэн зэрэг идэж болохгүй.

• Monitor-ийн pickup_forks(i) ба return_forks(i) үйлдэл, state[] (THINKING, HUNGRY, EATING), self[] condition variable.

• Pthreads mutex/condition variable ашиглана.

• Тоо ба хугацааны параметрийг өөрчилж болно, thinking/hungry/eating төлөв ба fork эзэмшилтийг дүрсэлнэ (багшийн UI шаардлага).

3. Онолын үндэс

Deadlock дөрвөн нөхцөлтэй: mutual exclusion, hold and wait, no preemption, circular wait. Naive шийдэлд philosopher бүр эхлээд зүүн fork-оо авч, дараа нь баруун fork-оо хүлээдэг тул бүгд зүүн fork-оо барьсан үед circular wait үүсч deadlock болно. Monitor шийдэл нь fork-ийг "хоёуланги нь нэг дор авах" атом үйлдэл болгодог (hold and wait-ийг устгана): philosopher зөвхөн хоёр хөрш нь идэхгүй байгаа үед EATING төлөвт орно. Starvation-ийг тус шийдэл бүрэн үгүйсгэдэггүй (хөрш нар ээлжлэн идвэл тухайн philosopher удаан хүлээж болзошгүй).

4. Системийн зохиомж

```
philosopher thread i:  loop { think(); pickup(i); eat(); putdown(i) }

monitor (mutex mon):
  pickup(i):   state[i]=HUNGRY; test(i); while(state[i]!=EATING) wait(self[i])
  putdown(i):  state[i]=THINKING; test(left); test(right)
  test(i):     if state[i]==HUNGRY && state[left]!=EATING && state[right]!=EATING
                   state[i]=EATING; signal(self[i])
```

Fork i нь philosopher i-ийн зүүн, philosopher (i+1) mod N-ийн баруун талд оршино.
Системд хоёр нэгж бий: src/philosophers.c (бодит Pthreads) ба

src/dining_philosophers.html (UI симуляц). Симуляц нь ижил state/test логикийг дискрет цагийн алхмаар ажиллуулна.

5. Хэрэгжилт

• Хэл: C (C99, gcc, -pthread); UI: HTML + JavaScript + SVG, гадаад сангүй.

• Нэг mutex + N condition variable, rand_r-ээр philosopher бүрийн тусдаа seed (давтагдах боломжтой).

• Инвариант шалгалт: pickup дотор хөрш нь идэж байвал violations++ ; нэгэн зэрэг идэж буй хамгийн их тоог (max_concurrent_eating) тоолно.

• Оролтыг шалгана: -n 2..64, -m≥1, -t/-e нь 0<min≤max; алдаа гарвал мессеж хэвлэж rc=2.

• UI: Start/Pause/Step/Reset, N, thinking/eating min/max, seed, хурд, алгоритм сонголт; буруу оролтод улаан мэдээлэл.

6. Туршилт ба UI дүрслэл

UI дээр өөрчилж болох зүйл: philosopher-ийн тоо (2–12), thinking/eating хугацаа, seed, хурд, алгоритм. Ажиглах зүйл: philosopher-ийн өнгө (Thinking = цэнхэр, Hungry = шар, Eating = ногоон), fork-ийн байрлал (хоосон үед ширээн дээр саарал, эзэмшсэн үед эзэмшигчийн хажууд эзэмшигчийн төлөвийн өнгөөр), хүснэгтийн хоол/хүлээлтийн тоо, event log. Naive горимд бүгд hungry болж зүүн fork-оо барьсан үед DEADLOCK анхааруулга гарна.

[Screenshot 1: Monitor, N=5] [Screenshot 2: Naive горимын deadlock] [Screenshot 3: N=8, өөр хугацаа] — Эдгээрийг UI-аас өөрсдөө авч энд оруулна уу.

7. Туршилтын арга

Орчин: Ubuntu 24 (Linux x86-64), gcc, GNU make. Хүлээгдсэн үр дүн: бүх philosopher MEALS удаа идэж дуусна (total = N×MEALS), violations = 0, max_concurrent_eating ≤ ⌊N/2⌋, програм зогсохгүй (deadlock-гүй). Алдаатай оролтод rc=2 ба тодорхой мессеж.

8. Үр дүн

Дараах хүснэгт нь ./philosophers -q -n N -m 5 -t 20,60 -e 20,60 -s 7 командын бодит гаралт:

N	Нийт хоол	Зөрчил	Нэгэн зэрэг идсэн max	Max хүлээлт (мс)	Нийт хугацаа (мс)
2	10/10	0	1	43	515
3	15/15	0	1	86	687
5	25/25	0	2	91	639
8	40/40	0	4	73	536
12	60/60	0	6	74	601

Автомат тестийн дүн (tests/results.txt): нийт 45 тест — T1–T7 үндсэн, T8 (30 удаа, N=7, seed 100–129) стресс, E1–E8 алдаа/захын оролт. Бүгд PASS.

Тест	Оролт	Хүлээгдсэн	Дүн
T1	N=5 m=3 t=20,60 e=20,40	total=15, violations=0	PASS
T2	N=2 m=5	total=10, нэг дор ≤1 идэнэ	PASS

Тест	Ролт	Хүлээгдсэн	Дүн
T4	N=12	deadlock-гүй дуусна	PASS
T5	N=64 (дээд хязгаар)	дуусна	PASS
T6	min=max (тогтмол хугацаа)	дуусна	PASS
E1/E2	N=1, N=65	rc=2 + мессеж	PASS
E5	-t 300,100	rc=2 + мессеж	PASS
E4	-m 0	rc=2	PASS

9. Хэлэлцүүлэг

N нэмэгдэхэд нэгэн зэрэг идэх philosopher-ийн дээд тоо ⌊N/2⌋-д ойртдог (N=12 үед 6) нь параллелизмын онолын дээд хязгаартай нийцнэ. Хүлээлтийн хугацаа нь санамсаргүй хугацаанаас хамаарна. Хязгаарлалт: (1) monitor шийдэл starvation-аас албан ёсоор хамгаалдаггүй; (2) UI нь бодит Pthreads биш дискрет симуляц, цаг хугацаа нь simulated; (3) C програм нь Naive горимыг хэрэгжүүлээгүй (deadlock-ийг зөвхөн UI-д харуулна); (4) UI-г автомат тестлээгүй, гараар шалгана; (5) хүснэгтэд нэг л ажиллалтын үр дүн, статистик дундаж биш.

10. Дүгнэлт

Сурах бичгийн monitor шийдлийг Pthreads-ээр хэрэгжүүлж, 45 тестээр зөв ажиллагааг баталгаажуулав. UI нь төлөв ба fork эзэмшлийг бодит логиктой нэгэн адил харуулж, deadlock-ийн механизмыг ойлгоход тусална.

GitHub дээрх ажиллагаа

[Гишүүн бүрийн commit, branch, issue-г энд бичнэ]

AI ба бэлэн код

Код ба тайлангийн эхний хувилбарыг AI (Claude)-ийн тусламжтай бэлтгэсэн. Алгоритм нь сурах бичгийн monitor шийдэл. Гишүүд шалгасан эсэхээ энд баталгаажуулна.

Ашигласан материал

Silberschatz, Galvin, Gagne. Operating System Concepts, Ch.7 (Synchronization Examples; Project 11). POSIX Threads (pthread_mutex, pthread_cond).